

The "CLI Bank ATM"

Create a command-line interface (CLI) program that simulates a simple Automated Teller Machine (ATM). The program must run continuously until the user chooses to "Exit" and must not crash if the user enters invalid data.

1. Technical Requirements (The "Rules")

1. **Variables:**
 - Initialize a variable named **balance** with a starting value (e.g., \$1,000, bonus if you can randomize this starting number).
 2. **The Loop:**
 - The program must use loop to keep the menu running. The program should only stop when the user selects the "Exit" option.
 3. **Conditionals:**
 - Create a menu with 4 options: Check Balance, Deposit, Withdraw, and Exit.
 - **Logic Check:** If the user tries to withdraw more money than they have, print a warning message (e.g., "Insufficient Funds") and do not deduct the money.
 4. **Exception Handling:**
 - If the program asks for a number (like a menu choice or a deposit amount) and the user types a word (e.g., "hello"), the program must **catch** the error, print "Invalid input, please enter a number," and restart the loop. It must not crash.
-

2. Sample Expected Output

```
None
--- WELCOME TO UPSKILL BANK ---
Current Balance: $1000

1. Deposit
2. Withdraw
3. Exit

Select an option: 1
Enter amount to deposit: 500
$500 deposited. New Balance: $1500
```

```
--- WELCOME TO UPSKILL BANK ---
...
Select an option: 2
Enter amount to withdraw: two hundred
Error: Please enter a valid number!

--- WELCOME TO UPSKILL BANK ---
...
Select an option: 2
Enter amount to withdraw: 5000
Error: Insufficient funds! You only have $1500.

--- WELCOME TO UPSKILL BANK ---
...
Select an option: 3
Thank you for banking with us. Goodbye!
```

3. "Extra Credit" Challenge

- Create an empty list `history = []`
- Every time a deposit or withdrawal happens, append a string to the list (e.g., `"Withdrew $50"`).
- Add a menu option "4. Print Statement" that uses a **for loop** to iterate through the list and print every transaction.